IPA ASSIGNMENT – 25

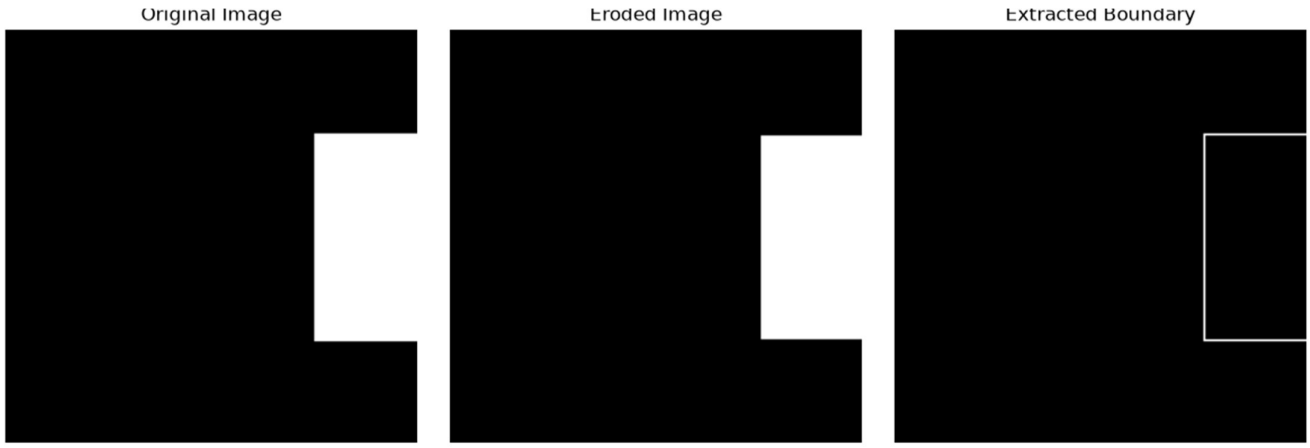
Name – Nishant Nahar

Roll No – 241551078

1. Boundary Detection

```
import cv2 import numpy as np
import matplotlib.pyplot as plt image = np.zeros((200, 200), dtype=np.uint8)
cv2.rectangle(image, (150, 50), (1520, 150), 2535, -1)
kernel = np.ones((3, 3), np.uint8)
eroded = cv2.erode(image, kernel, iterations=1)
boundary = cv2.subtract(image, eroded)
plt.figure(figsize=(12, 4)) plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray') plt.title("Original Image")
plt.axis("off") plt.subplot(1, 3, 2)
plt.imshow(eroded, cmap='gray') plt.title("Eroded Image")
plt.axis("off") plt.subplot(1, 3, 3) plt.imshow(boundary, cmap='gray')
plt.title("Extracted Boundary") plt.axis("off") plt.tight_layout() plt.show()
```

OUTPUT



2. Region Detection

```
import cv2 import numpy as np
import matplotlib.pyplot as plt image = np.zeros((200, 200), dtype=np.uint8)
cv2.circle(image, (10, 100), 150, 255, 3) filled = image.copy()
mask = np.zeros((202, 202), dtype=np.uint8)
cv2.floodFill(filled, mask, (10, 100), 255)
result = cv2.bitwise_or(image, filled) plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1) plt.imshow(image, cmap="gray") plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(filled, cmap="gray")
plt.title("After Flood Fill")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(result, cmap="gray")
plt.title("Region Filled")
plt.axis("off")
plt.tight_layout()
plt.show()
```

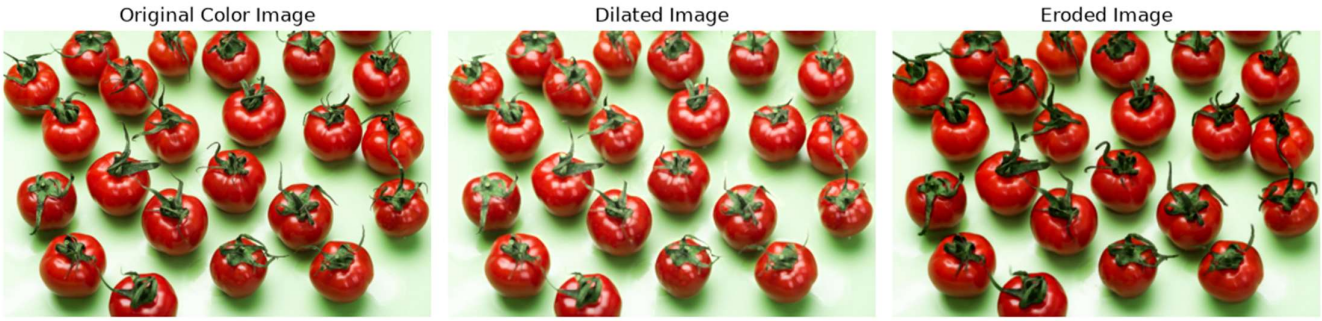
OUTPUT



3. Dilation and Erosion in Color Image

```
import cv2 import numpy as np
import matplotlib.pyplot as plt
image = cv2.imread("img_2.jpeg")
if image is None:raiseFileNotFoundError("Place image.jpg in the working directory")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
kernel = np.ones((5, 5), np.uint8)
dilated = cv2.dilate(image, kernel, iterations=1)
eroded = cv2.erode(image, kernel, iterations=1)
plt.figure(figsize=(12, 5)) plt.subplot(1, 3, 1)
plt.imshow(image) plt.title("Original Color Image")
plt.axis("off") plt.subplot(1, 3, 2)
plt.imshow(dilated) plt.title("Dilated Image")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(eroded)
plt.title("Eroded Image")
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT



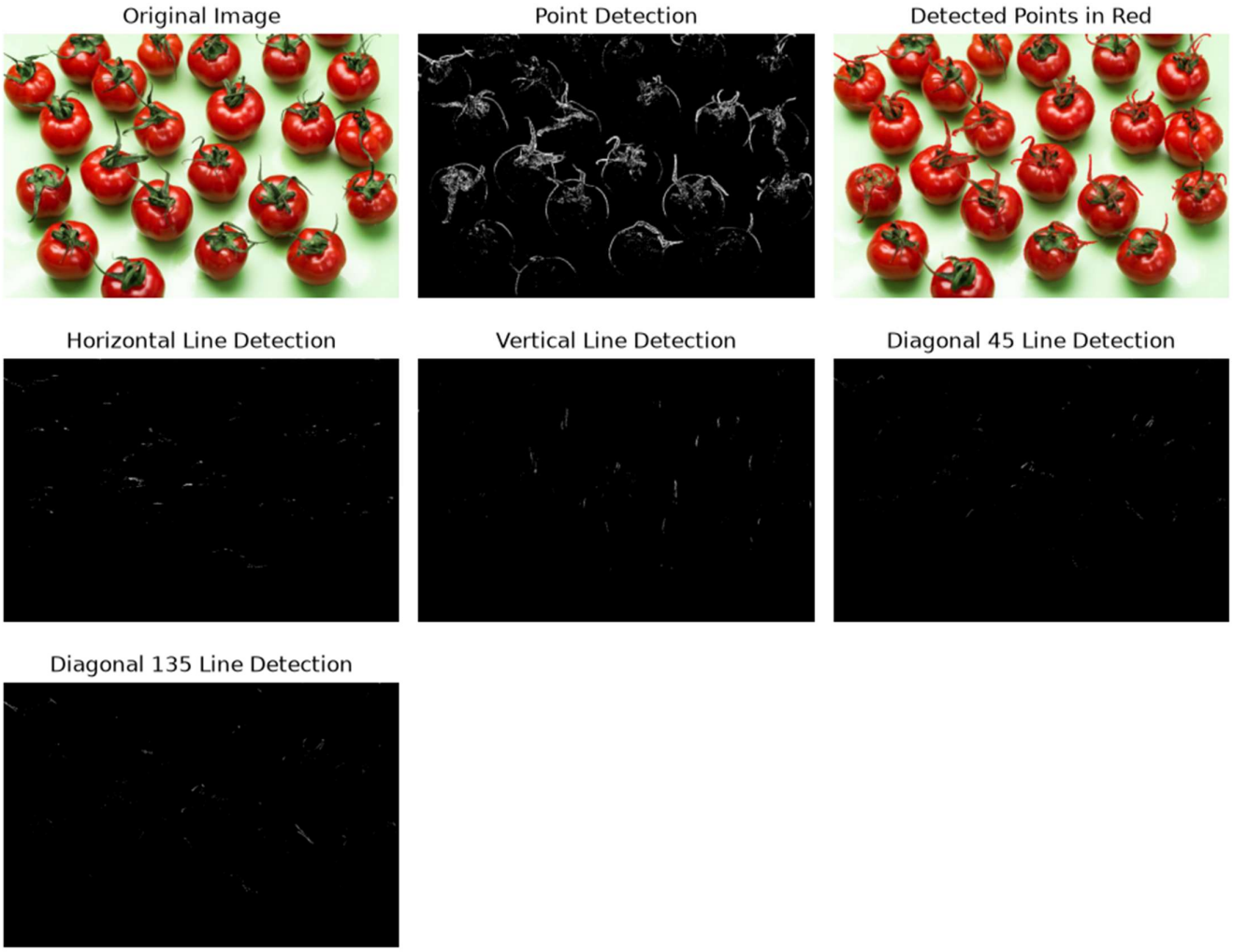
4. Line and Point Detection

```
import cv2 import numpy as np
import matplotlib.pyplot as plt
image = cv2.imread("img_2.jpeg")
```

```
if image is None:raise FileNotFoundError("Image img_1.jpeg not found in the working
directory")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
point_kernel = np.array([
    [-1, -1, -1],
    [-1,  8, -1],
    [-1, -1, -1]], dtype=np.float32)
point_response = cv2.filter2D(gray, cv2.CV_32F, point_kernel)
point_response = np.abs(point_response)
_, point_mask = cv2.threshold(point_response, 80, 255, cv2.THRESH_BINARY)
point_mask = point_mask.astype(np.uint8)
point_result = image.copy()
point_result[point_mask > 0] = [0, 0, 255]  # BGR red
horizontal_kernel = np.array([
    [-1, -1, -1],
    [ 2,  2,  2],
    [-1, -1, -1]], dtype=np.float32)
vertical_kernel = np.array([
    [-1,  2, -1],
    [-1,  2, -1],
    [-1,  2, -1]], dtype=np.float32)
diagonal_45_kernel = np.array([
    [-1, -1,  2],
    [-1,  2, -1],
    [ 2, -1, -1]], dtype=np.float32)
diagonal_135_kernel = np.array([
    [ 2, -1, -1],
    [-1,  2, -1],
    [-1, -1,  2]], dtype=np.float32)
kernels = [
    ("Horizontal Line Detection", horizontal_kernel),
    ("Vertical Line Detection", vertical_kernel),
    ("Diagonal 45 Line Detection", diagonal_45_kernel),
    ("Diagonal 135 Line Detection", diagonal_135_kernel)]
line_results = []
for name, kernel in kernels:
    response = cv2.filter2D(gray, cv2.CV_32F, kernel)
    response = np.abs(response)
    response=cv2.normalize(response,None,0,255,cv2.NORM_MINMAX).astype(np.uint8)
    _, detected = cv2.threshold(response, 100, 255, cv2.THRESH_BINARY)
    line_results.append((name, detected))
plt.figure(figsize=(10, 8)) plt.subplot(3, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB)) plt.title("Original Image")
plt.axis("off") plt.subplot(3, 3, 2) plt.imshow(point_mask, cmap="gray")
plt.title("Point Detection")
plt.axis("off")
plt.subplot(3, 3, 3)
plt.imshow(cv2.cvtColor(point_result, cv2.COLOR_BGR2RGB))
plt.title("Detected Points in Red")
plt.axis("off")
for i, (name, result) in enumerate(line_results):
    plt.subplot(3, 3, i + 4)
    plt.imshow(result, cmap="gray")
    plt.title(name)
    plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

OUTPUT

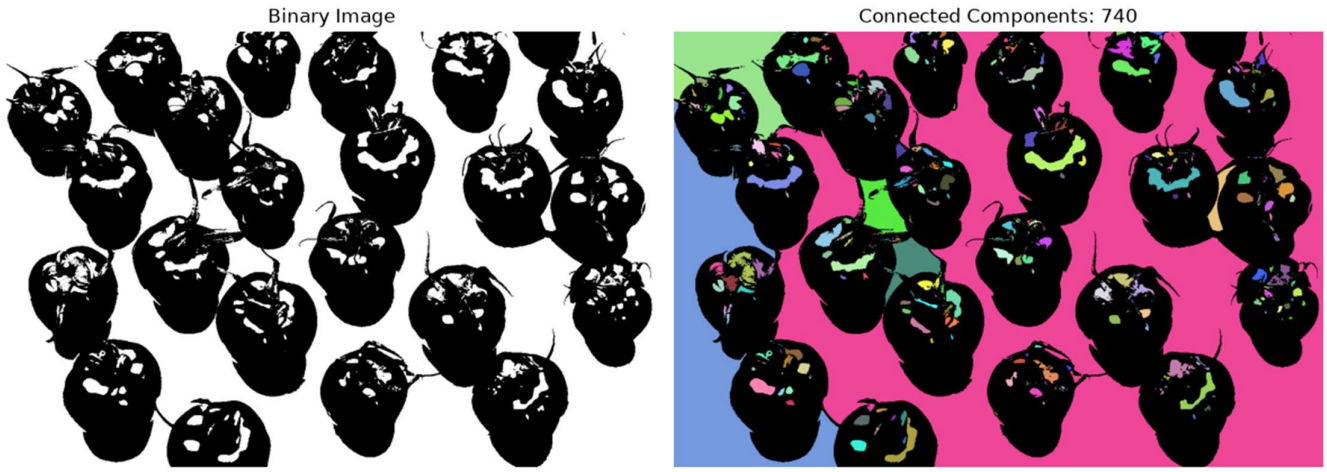


5. Connected Components

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_2.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None:raise FileNotFoundError("img_1.jpeg not found")
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(binary,
connectivity=8)
output = np.zeros((*labels.shape, 3), dtype=np.uint8)
np.random.seed(42)
for i in range(1, num_labels):
output[labels == i] = np.random.randint(50, 256, 3)
plt.figure(figsize=(12, 5)) plt.subplot(1, 2, 1)
plt.imshow(binary, cmap="gray")
plt.title("Binary Image") plt.axis("off")
plt.subplot(1, 2, 2) plt.imshow(output)
plt.title(f"Connected Components: {num_labels - 1}")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

OUTPUT



6. Thinning and Skeletonization

```
import cv2 import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_2.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None:  raise FileNotFoundError("img_1.jpeg not found")
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
thinning = cv2.ximgproc.thinning(binary)
skeleton = np.zeros_like(binary)
element = cv2.getStructuringElement(cv2.MORPH_CROSS, (3, 3)) work = binary.copy()
while cv2.countNonZero(work) > 0:
    eroded = cv2.erode(work, element)
    opened = cv2.dilate(eroded, element)
    residue = cv2.subtract(work, opened)
    skeleton = cv2.bitwise_or(skeleton, residue)
    work = eroded
plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.imshow(binary, cmap="gray")
plt.title("Original Binary Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(thinning, cmap="gray")
plt.title("Thinning")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(skeleton, cmap="gray")
plt.title("Skeletonization")
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT

